

Elaboración de Documentación Técnica y Manual

Actividad N°: 38 **Fecha:** 22/07/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Consolidar la documentación técnica final del proyecto, verificando que refleje el estado real del sistema después de todas las correcciones aplicadas durante el desarrollo, incorporar la aclaración sobre credenciales solicitada por el tutor en la validación de la Semana 7, y actualizar cada documento técnico contra el código fuente real para detectar inconsistencias antes del cierre.

2. Inventario de documentación técnica del proyecto

Documento	Contenido	Estado
README.md	Descripción general, inicio rápido, estructura del proyecto, endpoints, roles	✓ Vigente
SETUP.md	Guía de instalación y configuración paso a paso	✓ Vigente
ARQUITECTURA.md	Diagramas de arquitectura, base de datos, rutas API y frontend	✓ Vigente
CAMBIOS.md	Changelog de la versión 2.0 (Lumineers)	✓ Vigente
OPTIMIZACIONES_BACKEND.md	Detalle de optimizaciones de rendimiento aplicadas	✓ Vigente, referenciado en Semana 6
CANJE_MASIVO_Y_MEJORAS_UI.md	Detalle de la funcionalidad de canje masivo	✓ Vigente
ACTUALIZACIONES_TIEMPO_REAL.md	Detalle de la implementación de Socket.IO	✓ Vigente
EVIDENCIAS_PASANTIA/	Evidencia semanal completa del proceso de desarrollo (Semanas 1 a 8)	✓ Este ciclo de documentación

3. Aclaración incorporada: credenciales de acceso inicial del sistema

Tal como se acordó en la validación con el tutor empresarial (Semana 7, Día 34), se deja documentada de forma explícita y centralizada la forma correcta de generar el acceso inicial al sistema:

Único procedimiento válido para crear el primer usuario administrador:

```
cd backend
node src/scripts/setup.js ../../LUMINEERS.csv
```

Esto crea el usuario **sistema** / **sistema-inicial** (rol Jefe), además de importar los boletos del evento y configurar el Punto de Venta con sus localidades.

Importante: desde la corrección aplicada en la Semana 6, **config/database.js** ya **no** crea ningún usuario administrador automáticamente al arrancar el servidor. **setup.js** es la única fuente de creación del usuario inicial, evitando la inconsistencia detectada anteriormente (usuario **admin@shakira.com** duplicado con credenciales distintas a las documentadas).

4. Manual técnico de cierre (resumen ejecutivo para el equipo de la empresa)

Requisitos: Node.js 16+, cuenta de MongoDB Atlas, cuenta de Render (para producción).

Puesta en marcha (nuevo evento):

1. Configurar **backend/.env** con la URI de MongoDB (rotada según el hallazgo de la Semana 8, Día 36) y el **JWT_SECRET**.
2. Ejecutar **node src/scripts/setup.js <ruta-al-csv-del-evento>** para importar los boletos y crear el acceso inicial.
3. Iniciar backend (**npm start**) y frontend (**npm run build** + publicación del **dist**), o desplegar en Render siguiendo el procedimiento del Día 37.
4. Ejecutar el script **verify_deploy.sh** (Día 37) para confirmar que el sistema está operativo.
5. Entregar la guía de capacitación (Día 38) al personal de Staff e Impresor antes del evento.

Roles y accesos:

- **Jefe:** acceso total (dashboard, usuarios, puntos de venta, auditoría, canje individual y masivo).
- **Staff:** solo canje y búsqueda, restringido a su punto de trabajo.
- **Impresor:** solo impresión y reimpresión de tickets, con campo de motivo obligatorio para reimpresiones.

Soporte y trazabilidad:

- Todo canje, impresión y reimpresión queda registrado en el módulo de Auditoría (solo visible para Jefe), consultable por tipo, usuario y rango de fechas.

5. Verificación de documentos técnicos contra el estado real del código

Se realizó una revisión de cada documento técnico del inventario, comparando su contenido contra el estado real del código al cierre del proyecto, para detectar referencias desactualizadas o afirmaciones que ya no sean válidas tras las correcciones de las semanas anteriores:

5.1 README.md

Puntos revisados:

Afirmación en el documento	Estado en el código real	Acción
"El usuario administrador inicial se crea automáticamente al arrancar el servidor"	✗ Incorrecto: esta funcionalidad fue eliminada en la Semana 6; el usuario se crea con <code>setup.js</code>	Actualizar con la aclaración de la Sección 3
"El sistema utiliza el servicio <code>shakira-tickets</code> en los logs"	✗ Incorrecto: corregido a <code>canje-fft</code> en Semana 7	Actualizar referencia al nombre del servicio
Lista de endpoints de la API	✓ Coincide con las rutas registradas en <code>routes/</code>	Sin cambios
Descripción de roles (Jefe / Staff)	✓ Coincide con el middleware <code>authorize()</code>	Sin cambios

Correcciones aplicadas en `README.md`: párrafo de "primer acceso" actualizado para reflejar que el usuario se crea con `setup.js`; referencia al nombre del servicio de logging actualizada.

5.2 `SETUP.md`

Puntos revisados:

Afirmación en el documento	Estado en el código real	Acción
Instrucción de crear variable <code>MONGODB_URI</code> en <code>.env</code>	✓ Correcto; <code>.env.example</code> creado en Día 36 como referencia	Agregar mención al archivo <code>.env.example</code>
Instrucción de ejecutar <code>setup.js</code> para inicializar la base	✓ Correcto y es el único mecanismo válido	Sin cambios
Versión de Node.js requerida (16+)	✓ Compatible con las dependencias del <code>package.json</code>	Sin cambios

Correcciones aplicadas en `SETUP.md`: párrafo de configuración actualizado para referenciar `backend/.env.example`.

5.3 `ARQUITECTURA.md`

Puntos revisados:

Afirmación en el documento	Estado en el código real	Acción
Diagrama de flujo de autenticación (JWT)	✓ Coincide con la implementación real	Sin cambios
Descripción del modelo <code>User</code> con campos <code>primerAcceso</code> y <code>puntoTrabajo</code>	✓ Coincide con <code>User.js</code>	Sin cambios
Descripción del modelo <code>AuditLog</code>	✓ Coincide con <code>AuditLog.js</code>	Sin cambios

Afirmación en el documento	Estado en el código real	Acción
Mención de <code>createDefaultAdmin()</code> en <code>database.js</code>	✗ Función eliminada en Semana 6	Eliminar referencia

Correcciones aplicadas en ARQUITECTURA.md: referencia a `createDefaultAdmin()` eliminada del diagrama de inicialización.

5.4 CAMBIOS.md

Puntos revisados:

Afirmación en el documento	Estado en el código real	Acción
Lista de cambios de la versión 2.0 (Lumineers)	✓ Coincide con las funcionalidades implementadas	Sin cambios
Referencia a la base de datos "Shakira" como origen	✓ Mencionada como contexto histórico, no como configuración activa	Sin cambios

Resultado: sin correcciones necesarias.

5.5 Documentos de funcionalidades (OPTIMIZACIONES_BACKEND.md, CANJE_MASIVO_Y_MEJORAS_UI.md, ACTUALIZACIONES_TIEMPO_REAL.md)

Revisados en busca de referencias al nombre de servicio del logger (`shakira-tickets`), al fallback de localidades eliminado, y a la doble creación de administrador. **Resultado:** ninguno de estos documentos describe esas funcionalidades de forma que requiera actualización; describen la funcionalidad final implementada sin mencionar los mecanismos corregidos.

6. Registro de deuda técnica al cierre del proyecto

Como complemento del manual técnico, se deja un registro consolidado y priorizado de la deuda técnica conocida al momento del cierre, para orientar la planificación de futuras iteraciones:

#	Descripción	Impacto potencial	Prioridad
DT-01	Auditoría potencialmente duplicada: rutas de canje y reimpresión tienen tanto el middleware <code>auditLogger</code> como el controlador creando registros en <code>AuditLog</code> . Cada operación puede generar dos filas en la tabla de auditoría — una completa y una incompleta	Tabla de auditoría con registros redundantes; filtros por tipo pueden devolver resultados duplicados	Media
DT-02	El endpoint de canje no valida que el ticket pertenezca al punto de trabajo del Staff autenticado a nivel de API. La interfaz de Staff solo muestra tickets de su punto, pero la restricción no existe en el backend	Un Staff con un cliente de API personalizado podría canjear tickets fuera de su punto de trabajo	Media

#	Descripción	Impacto potencial	Prioridad
DT-03	Credenciales de MongoDB en historial de Git de <code>render.yaml</code> (archivo modificado, pero el historial conserva el valor anterior). Requiere rotación de contraseña en MongoDB Atlas	Acceso no autorizado a la base de datos si el repositorio es público o si el historial es accedido	Alta (acción inmediata pendiente del tutor empresarial)
DT-04	No existe suite de pruebas automatizadas. Toda la verificación fue manual o por inspección de código	Mayor riesgo de regresiones en iteraciones futuras; más tiempo de QA por ciclo	Media
DT-05	El script <code>setup.js</code> no valida el formato del CSV antes de intentar importarlo. Un CSV mal formado podría causar un error no informativo durante la inicialización	Dificultad para diagnosticar errores de importación en producción	Baja
DT-06	El rol <code>Impresor</code> no está incluido en el enum <code>rol</code> del modelo <code>User.js</code> (<code>['jefe', 'staff']</code>), pese a que el resto del sistema (rutas, frontend, guías) lo contempla. Crear un usuario con <code>rol: 'impresor'</code> sería rechazado por la validación de Mongoose	Imposibilidad de usar el rol Impresor sin modificar el schema	Alta (bloquea una funcionalidad documentada)
DT-07	Existen dos colecciones de MongoDB con los mismos tickets (<code>FechaUno</code> y <code>Lumineers_Canje</code>). <code>ticketController.js</code> opera sobre <code>Lumineers_Canje</code> (vía <code>req.TicketModel</code>), mientras que <code>puntoVentaController.js</code> importa el modelo <code>Ticket</code> fijo apuntado a <code>FechaUno</code> . Los canjes se escriben en una colección y se leen desde la otra, haciendo que el estado de canje sea invisible en el flujo normal de Staff y Jefe	Los canjes quedan registrados en Auditoría pero la interfaz los muestra como no canjeados; los contadores del dashboard no reflejan el estado real	Alta (afecta la operación en producción)
DT-08	El enrutamiento de eventos Socket.IO en <code>canjeTicket</code> y <code>bulkCanjeTickets</code> emite a <code>punto-venta-\${ticket.puntoVenta}</code> y <code>staff-\${req.user.puntoTrabajo}</code> , pero <code>ticket.puntoVenta</code> no existe como campo en el schema de <code>Ticket</code> (el campo real es <code>Seat</code> /localidad), y <code>req.user.puntoTrabajo</code> es <code>undefined</code> para el rol Jefe. Las salas de WebSocket resultan siempre <code>undefined</code> , por lo que ningún cliente recibe la actualización en tiempo real al realizarse un canje	Los canjes no se propagan en tiempo real entre puntos de venta ni entre pestañas del mismo usuario	Media (la funcionalidad existe pero no opera en producción)

7. Conclusiones del día

La documentación técnica del proyecto queda consolidada y verificada contra el estado real del código, con tres documentos actualizados ([README.md](#), [SETUP.md](#), [ARQUITECTURA.md](#)) para eliminar referencias a funcionalidades ya corregidas. Se incorporó explícitamente la aclaración sobre el flujo correcto de creación del usuario administrador solicitada por el tutor empresarial, y se completó el manual técnico de cierre con el nuevo script de verificación del Día 37. Se deja un registro priorizado de 8 items de deuda técnica para guiar futuras iteraciones, incluyendo dos hallazgos de alto impacto en producción: la divergencia de colecciones de MongoDB que hace invisible el estado de canje en la interfaz, y el enrutamiento incorrecto de Socket.IO que impide la propagación en tiempo real de los canjes.

Observaciones: Tres documentos técnicos actualizados tras verificación vs. código; 8 items de deuda técnica registrados (2 de ellos con impacto directo en producción); manual de cierre consolidado.